

Linux for DevOps

Complete Interview Q&A Guide

Structured answers for DevOps Engineer · Cloud Engineer · SRE interviews

Tier 1 — Must Know	Tier 2 — Very Common	Tier 3 — Scenarios
Permissions · Processes Networking · Logs File System · grep/awk/sed Disk · Users · Scripting	Systemd · Packages SSH & Security CPU/Memory Debug Log Rotation	Disk Full · High CPU High Memory · Service Fail App Unreachable · SSH Fail Performance · Log Analysis

Q1

What is Linux?

Linux is a free, open-source operating system kernel created by Linus Torvalds in 1991. It is the core that manages hardware resources and allows software to run on a computer.

- It is open-source — anyone can view, modify, and distribute the code.
- It powers servers, cloud platforms, containers (Docker/Kubernetes), Android devices, and supercomputers.
- In DevOps, almost all production servers run on a Linux-based OS.

Q2

What is the Linux kernel?

The kernel is the innermost layer of the OS that directly talks to hardware. It manages:

- CPU scheduling — decides which process runs and when.
- Memory management — allocates and frees RAM.
- Device drivers — communicates with hardware (disk, network card, etc.).
- System calls — provides an interface for programs to request kernel services.

Q3

Difference between Linux and Unix?

Unix is a proprietary OS developed in the 1970s at Bell Labs. Linux is inspired by Unix but written from scratch and is open-source.

- Unix: commercial, proprietary (e.g., AIX, HP-UX, Solaris).
- Linux: free, open-source, community-driven.
- Both share similar commands and POSIX standards.

Q4

What are Linux distributions?

A Linux distribution (distro) bundles the Linux kernel with software, package managers, and desktop environments for a specific purpose.

- Ubuntu / Debian — developer-friendly, widely used in cloud.
- CentOS / RHEL / AlmaLinux — enterprise, stable, used in production.
- Amazon Linux — optimised for AWS EC2.
- Alpine Linux — tiny image used in Docker containers.

Q5

What happens when a Linux system boots?

- BIOS/UEFI — firmware initialises hardware and finds the bootloader.
- Bootloader (GRUB) — loads the Linux kernel into memory.
- Kernel initialisation — kernel decompresses, detects hardware, mounts root filesystem.
- Init system (systemd) — starts as PID 1 and launches all system services.
- Login prompt — system is ready for user login.

Q6

What is a shell? Difference between Bash and sh?

A shell is a command-line interpreter — it reads commands you type and executes them.

- sh (Bourne Shell) — the original, minimal shell. Available on every Unix/Linux system.
- Bash (Bourne Again Shell) — extended version of sh with more features: arrays, string manipulation, tab completion, history.
- For DevOps scripting, Bash is the standard choice.

```
echo $SHELL # shows your current shell bash --version # check Bash version
```

SECTION 02

Linux File System Hierarchy

Q7

Explain the Linux file system hierarchy.

Everything in Linux is organised under a single root directory '/'. Each subdirectory has a specific purpose:

Directory	Purpose
/etc	Configuration files for system and applications (e.g., /etc/nginx/nginx.conf)
/var	Variable data — logs (/var/log), mail, databases, spool files
/home	Personal directories for regular users (e.g., /home/ubuntu)
/tmp	Temporary files, cleared on reboot — world-writable
/proc	Virtual filesystem — runtime kernel & process info (e.g., /proc/cpuinfo)
/dev	Device files — represents hardware (e.g., /dev/sda for disk)
/opt	Optional/third-party software installed manually
/usr	User programs and libraries (/usr/bin, /usr/lib, /usr/share)
/bin	Essential user binaries needed in single-user mode (ls, cp, mv)
/sbin	System binaries for root/admin tasks (fdisk, reboot, ifconfig)
/boot	Kernel files and bootloader (vmlinuz, initrd, grub)
/lib	Shared libraries needed by /bin and /sbin binaries
/mnt	Mount point for temporarily mounted filesystems
/media	Mount point for removable media (USB, CD-ROM)

SECTION 03

File Permissions

Q8

What are Linux permissions? Explain read, write, execute.

Every file/directory has permissions for three entities: Owner (u), Group (g), Others (o).

- r (read=4) — view file contents or list directory.
- w (write=2) — modify file contents or create/delete files in directory.
- x (execute=1) — run a file as a program or enter a directory.

```
ls -la file.sh -rwxr-xr-- 1 ubuntu devops 1234 Jun 09 10:00 file.sh # ^
owner=rwx(7) group=r-x(5) others=r--(4)
```

Q9

What does 755, 777, and 644 mean?

- 755 → Owner: rwx (7), Group: r-x (5), Others: r-x (5). Common for scripts and web server directories.
- 777 → Everyone has full read/write/execute. Avoid in production — serious security risk.
- 644 → Owner: rw- (6), Group: r-- (4), Others: r-- (4). Common for config files and web content.

```
chmod 755 deploy.sh # owner can run; others can only read/execute
chmod 644 config.yaml # owner can edit; others read-only
chmod +x script.sh # add execute for all
```

Q10

What is chmod, chown, chgrp, and umask?

- chmod — change file permissions (e.g., chmod 755 file.sh).
- chown — change file owner and group (e.g., chown ubuntu:devops app.py).
- chgrp — change only the group (e.g., chgrp developers /var/app).
- umask — sets the default permission mask for newly created files. Default umask 022 means new files get 644 and directories get 755.

```
umask # view current umask (e.g., 0022)
umask 027 # new files: 640, new dirs: 750
```

SECTION 04

Users and Groups

Q11

What is sudo? Difference between sudo and su?

- sudo — runs a single command as root. Logs the action. Requires the user to be in the sudoers file. Safer.
- su — switches to another user (default: root) for the whole session. Requires the target user's password.

```
sudo apt update # run one command as root
sudo -i # open root shell via sudo
su - # switch to root (needs root password)
```

Q12

How do you create, delete, and manage users?

```
useradd -m -s /bin/bash john # create user with home dir
passwd john # set password
usermod -aG docker john # add to docker group
userdel -r john # delete user + home dir
usermod -L john # lock account
id john # view user info
groups john # view groups
```

Q13

What is a process? PID? Zombie? Orphan?

- Process — a running instance of a program. Has its own PID, memory space, file descriptors.
- PID — Process ID, a unique integer assigned by the kernel.
- Zombie process — a process that has finished but its parent hasn't read its exit status yet. Shows as 'Z' in ps. Wastes a PID entry.
- Orphan process — a process whose parent has died. Adopted by init (PID 1 / systemd).
- Daemon — a background process with no terminal (e.g., sshd, nginx, crond).

Q14

What is process priority and nice value?

Priority determines how much CPU time a process gets. Nice value ranges from -20 (highest priority) to +19 (lowest). Lower nice = more CPU.

```
nice -n 10 ./backup.sh # start with lower priority
renice -n 5 -p 1234 # change priority of running PID
ps -eo pid,ni,comm | head -20 # view nice values
```

Q15

How do you investigate a process consuming 100% CPU?

```
top # real-time view; press P to sort by CPU
htop # interactive; easier to read
ps aux --sort=-%cpu # static list sorted by CPU
pidstat -u 1 5 # CPU per process, 5 samples
lsof -p # open files for that process
strace -p # system calls (advanced)
```

■ Press 'k' in top to kill a process directly. Use kill -9 only as last resort.

Command	Purpose
<code>ps aux</code>	List all processes with user, CPU, memory
<code>top</code> / <code>htop</code>	Real-time process monitor (htop is interactive)
<code>kill -15</code>	Graceful termination (SIGTERM)
<code>kill -9</code>	Forced kill (SIGKILL) — use only when -15 fails
<code>killall nginx</code>	Kill all processes named nginx
<code>nice</code> / <code>renice</code>	Set or change process priority
<code>bg</code> / <code>fg</code>	Move job to background / foreground
<code>nohup cmd &</code>	Run command immune to hangups, in background

Q16

What is an IP, DNS, subnet, gateway, NAT, CIDR?

- IP address — unique identifier for a device on a network (IPv4: 192.168.1.10).
- DNS — Domain Name System, translates domain names to IP addresses (google.com → 142.250.x.x).
- Subnet — a logical subdivision of a network. Subnet mask (e.g., 255.255.255.0 or /24) defines it.
- Gateway — the router IP that forwards traffic to other networks / the internet.
- NAT — Network Address Translation, maps private IPs to a public IP. Used in home routers and AWS VPCs.
- CIDR — Classless Inter-Domain Routing notation (e.g., 10.0.0.0/16 = 65,536 addresses).

Q17

What is TCP vs UDP?

- TCP — connection-oriented, reliable, ordered delivery, acknowledgements. Used by HTTP, SSH, databases.
- UDP — connectionless, faster, no guarantee of delivery. Used by DNS, VoIP, video streaming, monitoring.

Q18

Application is not reachable — which commands would you use?

```
# 1. Basic connectivity ping # is host alive? traceroute # where does traffic stop? # 2. DNS resolution nslookup app.example.com dig app.example.com # 3. Is the port open on the server? ss -tlnp | grep 8080 # is app listening? telnet 8080 # can you reach the port? curl -v http://:8080 # full HTTP check # 4. Firewall / security groups sudo iptables -L -n # local firewall rules # 5. Check server logs tail -f /var/log/nginx/error.log
```

Command	Purpose
ping	Check if host is reachable (ICMP)
traceroute / mtr	Trace network path to a host
nslookup / dig	DNS lookup — resolve domain to IP
curl -v URL	Make HTTP request with verbose output
wget URL	Download file from URL
ssh user@host	Secure Shell remote login
scp src dst	Secure copy file over SSH
ss -tlnp	Show listening TCP ports with PID
netstat -tulnp	Same as ss (older systems)
ip addr / ifconfig	Show network interfaces and IP addresses
ip route	Show routing table

SECTION 07

Disk and Storage

Q19

How do you check disk usage? What is LVM? What is swap?

- `df -h` — shows free/used space on all mounted filesystems (human-readable).
- `du -sh /path` — shows total size of a specific directory.
- `lsblk` — shows block devices (disks, partitions) in tree format.
- LVM (Logical Volume Manager) — allows flexible resizing of disk partitions without downtime. You can add new disks and extend volumes online.
- Swap — disk space used as virtual RAM when physical RAM is full. Prevents OOM (Out of Memory) crashes but is much slower than RAM.

```
df -h # disk space per filesystem du -sh /var/log/* # size of each log directory
lsblk # list all disks and partitions free -h # RAM and swap usage swapon --show #
active swap devices
```

Q20

How do you identify large files filling up disk?

```
du -ah / | sort -rh | head -20 # top 20 largest files/dirs find / -size +100M
-type f # files larger than 100MB ncdu / # interactive disk usage (install ncdu)
```

■ In Docker environments, docker system df and docker system prune are common culprits for disk filling up.

SECTION 08

Log Management

Q21

Where are Linux logs stored? Key log files?

- `/var/log/syslog` — general system messages (Ubuntu/Debian).
- `/var/log/messages` — general messages (RHEL/CentOS).
- `/var/log/auth.log` — authentication, sudo, SSH login attempts.
- `/var/log/kern.log` — kernel messages.
- `/var/log/nginx/` — nginx access and error logs.
- `/var/log/journal/` — systemd journal (binary format, read with `journalctl`).

Q22

How do you monitor and search logs?

```
tail -f /var/log/syslog # live log stream grep 'ERROR' /var/log/app.log # search
for errors grep -i 'fail' /var/log/auth.log # case-insensitive search journalctl
-u nginx -f # follow nginx service logs journalctl --since '1 hour ago' # logs
from last hour less +F /var/log/app.log # less with live follow (Ctrl+C to scroll)
```

Q23

What is log rotation? What is logrotate?

Log rotation automatically compresses, renames, and deletes old log files to prevent disks from filling up.

- logrotate is the tool that manages this. Configuration is in `/etc/logrotate.conf` and `/etc/logrotate.d/`.
- Common settings: daily/weekly rotation, keep last 7 copies, compress old logs, run postrotate scripts.

```
# Example /etc/logrotate.d/nginx /var/log/nginx/*.log { daily missingok rotate 14
compress postrotate nginx -s reopen endsript }
```

Q24

What is grep? Difference between grep and egrep?

- grep searches for patterns in files or streams using basic regular expressions.
- egrep (or grep -E) supports extended regular expressions — allows +, ?, |, () without escaping.

```
grep 'ERROR' app.log # simple text match grep -n 'ERROR' app.log # show line numbers
grep -r 'config' /etc/ # recursive search grep -v 'DEBUG' app.log # exclude lines with DEBUG
egrep 'ERROR|WARN' app.log # match ERROR or WARN
```

Q25

What is awk?

awk is a powerful text processing tool that works column by column. It splits each line into fields (\$1, \$2, ...).

```
awk '{print $1}' access.log # print 1st column (IP address) awk '{print $1, $9}' access.log # print IP and status code
awk '$9 == 404 {print $1}' access.log # print IPs with 404 errors awk '{sum += $NF} END {print sum}' f # sum last column
```

Q26

What is sed?

sed (stream editor) is used for find-and-replace, deletion, and transformation of text streams.

```
sed 's/dev/prod/g' config.yaml # replace all 'dev' with 'prod' sed -i 's/dev/prod/g' config.yaml # in-place edit of the file
sed '/^#/d' config.yaml # delete comment lines sed -n '10,20p' file.log # print only lines 10-20
```

Q27

What is cut, sort, uniq, wc, xargs?

- cut — extract specific columns/fields from each line.
- sort — sort lines alphabetically or numerically.
- uniq — remove or count duplicate lines (must be sorted first).
- wc — count lines (-l), words (-w), characters (-c).
- xargs — build commands from stdin output (e.g., pipe find output to rm).

```
cut -d',' -f1 data.csv # get first CSV column cat access.log | awk '{print $1}' | sort | uniq -c | sort -rn | head -n 1 # top IPs by request count
wc -l /var/log/app.log # count log lines find . -name '*.tmp' | xargs rm # delete all .tmp files
```

Q28

What is a shebang? How do variables work?

- Shebang (#!) is the first line of a script that tells the OS which interpreter to use.
- Variables: no spaces around =. Reference with \$VAR or \${VAR}.
- Positional parameters: \$1, \$2 ... are arguments passed to the script. \$0 is the script name. \$# is the count.

```
#!/bin/bash ENV='production' # assign variable echo "Deploying to $ENV" # use variable echo "Script: $0" echo "Args: $# Args passed, first is: $1"
```

Q29

How do conditionals and loops work in Bash?

```
# If-else if [ $ENV == 'production' ]; then echo 'PROD: be careful' elif [ $ENV == 'staging' ]; then echo 'Staging OK' else echo 'Unknown env' fi # For loop for server in web1 web2 web3; do ssh $server 'sudo systemctl restart nginx' done # While loop while true; do ping -c1 google.com && break sleep 5 done
```

Q30

How do you debug a Bash script?

- bash -x script.sh — prints each command before executing it (trace mode).
- set -x inside script — enable trace from that point.
- set -e — exit immediately on any error.
- set -u — treat unset variables as errors.
- set -o pipefail — catch errors in pipes (not just the last command).

```
#!/bin/bash set -euo pipefail # recommended for production scripts set -x # enable debug tracing
```

SECTION 11

Systemd and Services

Q31

What is systemd? What is a unit file?

systemd is the init system (PID 1) used by most modern Linux distros. It starts, stops, and monitors all services.

- Unit file — a configuration file that defines how a service should run. Located in /etc/systemd/system/ or /lib/systemd/system/.

```
# Example unit file: /etc/systemd/system/myapp.service [Unit] Description=My Application After=network.target [Service] User=appuser ExecStart=/usr/bin/python3 /opt/app/server.py Restart=always [Install] WantedBy=multi-user.target
```

Command	Purpose
<code>systemctl start nginx</code>	Start the nginx service
<code>systemctl stop nginx</code>	Stop the nginx service
<code>systemctl restart nginx</code>	Restart (stop + start)
<code>systemctl reload nginx</code>	Reload config without downtime

<code>systemctl status nginx</code>	Show status, last logs, PID
<code>systemctl enable nginx</code>	Auto-start on boot
<code>systemctl disable nginx</code>	Remove from auto-start
<code>journalctl -u nginx -f</code>	Follow live logs for nginx
<code>journalctl -u nginx -n 50</code>	Last 50 log lines for nginx
<code>systemctl list-units</code>	List all active units

SECTION 12

Linux Security

Q32

How do you secure SSH access?

- Disable password authentication — use SSH key pairs only.
- Disable root login — set `PermitRootLogin no` in `/etc/ssh/sshd_config`.
- Change default port — reduces automated scan attempts (e.g., Port 2222). Not a security solution by itself.
- Use fail2ban — automatically bans IPs after multiple failed login attempts.
- Restrict users — use `AllowUsers` or `AllowGroups` directives.

```
# /etc/ssh/sshd_config PermitRootLogin no PasswordAuthentication no AllowUsers
ubuntu deploy Port 22 # Generate SSH key pair (on client machine) ssh-keygen -t
ed25519 -C 'devops@company.com' # Copy public key to server ssh-copy-id
user@server
```

Q33

What is SELinux and AppArmor?

- SELinux (Security-Enhanced Linux) — mandatory access control system developed by NSA. Used on RHEL/CentOS. Every process and file has a security label. Even root is restricted.
- AppArmor — similar MAC system used on Ubuntu/Debian. Easier to configure than SELinux. Profiles restrict what each application can do.
- Principle of Least Privilege — processes should only have the minimum permissions they need. Critical in production environments.

SECTION 13

Troubleshooting Scenarios

■ Scenario: Disk is 100% Full

- Step 1:** `df -h` — identify which filesystem is full.
- Step 2:** `du -ah / | sort -rh | head -20` — find the largest files/directories.
- Step 3:** Check `/var/log/` for oversized log files. Truncate: `> /var/log/bigfile.log`
- Step 4:** Check Docker: `docker system df`; clean with `docker system prune -a`
- Step 5:** Check `/tmp` for large temporary files: `du -sh /tmp/*`
- Step 6:** Rotate logs immediately: `logrotate -f /etc/logrotate.conf`
- Step 7:** If still full, check for deleted-but-open files: `lsdf | grep deleted | awk '{print $7}' | sort -rn`

■ Scenario: CPU Usage is 100%

- Step 1:** `top` (press P) or `htop` — find the process consuming the most CPU.
- Step 2:** `ps aux --sort=-%cpu | head -10` — static sorted list.
- Step 3:** Check if it's a runaway loop: `strace -p` to see what it's doing.
- Step 4:** Check load average: `uptime` — if load > number of CPUs, system is overloaded.
- Step 5:** Check for CPU-stealing in VMs: `top` — look at '%st' (steal time).
- Step 6:** If safe, kill the process: `kill -15`, then `kill -9` if needed.
- Step 7:** Review app logs for root cause before restarting.

■ Scenario: High Memory Usage

- Step 1:** `free -h` — check total, used, free, and swap usage.
- Step 2:** `top` (press M) — sort processes by memory consumption.
- Step 3:** `ps aux --sort=-%mem | head -10` — top memory processes.
- Step 4:** Check for memory leak: `watch -n 2 'ps aux --sort=-%mem | head -5'`
- Step 5:** If swap is heavily used, the system is under memory pressure — may need more RAM or optimisation.
- Step 6:** Check OOM killer activity: `dmesg | grep -i 'out of memory'`

■ Scenario: Application Not Accessible

- Step 1:** Check if the service is running: `systemctl status appname`
- Step 2:** Check if it's listening on the right port: `ss -tlnp | grep`
- Step 3:** Check connectivity from client: `ping`, `curl -v http://`
- Step 4:** Check DNS resolution: `dig app.example.com`
- Step 5:** Check firewall rules: `iptables -L -n` or cloud security group settings.
- Step 6:** Check application logs: `journalctl -u appname -n 100` or `/var/log/app/error.log`
- Step 7:** Check system resources (disk, memory, CPU) — app may have crashed due to resource exhaustion.

■ Scenario: SSH Access Stops Working

- Step 1:** Try accessing the server via console (AWS EC2 Session Manager, cloud console).
- Step 2:** Check if sshd is running: `systemctl status sshd`
- Step 3:** Check sshd logs: `journalctl -u sshd -n 50`
- Step 4:** Verify firewall allows port 22: `iptables -L -n | grep 22` or check cloud security groups.
- Step 5:** Check if disk is full (sshd can fail to write): `df -h`
- Step 6:** Check `/etc/ssh/sshd_config` for recent (accidental) changes.
- Step 7:** If config was changed, test with: `sshd -t` (test mode, reports errors).

■ Scenario: Service Keeps Restarting

- Step 1:** Check service status: `systemctl status servicename` — note the exit code.
- Step 2:** Read recent logs: `journalctl -u servicename -n 100`
- Step 3:** Check if it's OOM killed: `dmesg | grep -i 'killed process'`
- Step 4:** Check for port conflict: `ss -tlnp | grep` (another process may own the port).
- Step 5:** Check config file syntax: `nginx -t`, `apache2ctl -t`, etc.
- Step 6:** Look at the unit file `Restart=` setting — it may be configured to restart too aggressively.
- Step 7:** Check app logs in `/var/log/` for the actual error.

SECTION 14

Linux in DevOps Context

Q34

How is Linux used in Docker? What are namespaces and cgroups?

Docker containers are NOT VMs — they share the host Linux kernel. Docker uses Linux kernel features to isolate containers:

- Namespaces — provide isolation of resources. Each container gets its own: PID namespace (processes), NET namespace (network interfaces), MNT namespace (filesystem mounts), UTS namespace (hostname), IPC namespace (inter-process communication).
- cgroups (control groups) — limit and account for resource usage (CPU, memory, I/O) per container. This is how `'docker run --memory=512m'` works.
- This is why containers are lightweight — no separate kernel, just isolated processes on the host kernel.

```
# View namespaces for a running container lsns | grep # View cgroup limits cat /sys/fs/cgroup/memory/docker//memory.limit_in_bytes
```

Q35

How does Kubernetes interact with Linux?

- Each Kubernetes node is a Linux server. Kubernetes uses the same Linux primitives (namespaces, cgroups) as Docker.
- kubelet — the Kubernetes node agent — runs as a systemd service on each worker node.
- Container runtime (containerd/CRI-O) — creates and manages containers using Linux kernel features.
- kube-proxy — uses iptables/ipvs rules to implement Kubernetes Services (load balancing).

```
systemctl status kubelet # check kubelet on a node journalctl -u kubelet -f #  
debug Kubernetes node issues kubectl exec -it pod -- bash # get shell into a  
container
```

Q36

How do you monitor Linux servers in a DevOps environment?

- System metrics — CPU, memory, disk I/O, network: collected by node_exporter (Prometheus).
- Logs — shipped to ELK stack (Elasticsearch, Logstash, Kibana) or Loki via Fluentd/Promtail.
- Alerts — Prometheus Alertmanager or CloudWatch Alarms (AWS) trigger PagerDuty/Slack notifications.
- APM — application performance via Datadog, New Relic, or Jaeger.
- Key metrics to monitor: CPU usage, memory usage, disk I/O wait, network errors, OOM events, service up/down.

SECTION 15

Package Management

Q37

APT (Ubuntu/Debian) vs YUM/DNF (RHEL/CentOS)?

- APT manages .deb packages. YUM/DNF manages .rpm packages.
- apt update refreshes the package list; apt upgrade installs newer versions.
- dnf is the modern replacement for yum on RHEL 8+/Fedora — faster and better dependency resolution.

```
# Ubuntu / Debian apt update && apt upgrade -y # refresh + upgrade all apt install  
-y nginx # install package apt remove nginx # remove package apt search nginx #  
search dpkg -l | grep nginx # verify installed # RHEL / CentOS / Amazon Linux yum  
install -y nginx dnf install -y nginx # modern equivalent rpm -qa | grep nginx #  
verify installed
```

CATEGORY	COMMAND	WHAT IT DOES
Permissions	<code>chmod 755 f</code>	<code>rwxr-xr-x</code>
Permissions	<code>chown user:grp f</code>	Change owner
Process	<code>ps aux</code>	List all processes
Process	<code>kill -9 PID</code>	Force kill
Process	<code>nice -n 10 cmd</code>	Lower priority
Disk	<code>df -h</code>	Filesystem usage
Disk	<code>du -sh /path</code>	Dir size
Disk	<code>lsblk</code>	Block devices
Network	<code>ss -tlnp</code>	Listening ports
Network	<code>curl -v URL</code>	HTTP debug
Network	<code>dig domain</code>	DNS lookup
Logs	<code>tail -f file</code>	Live log stream
Logs	<code>journalctl -u svc -f</code>	Service logs
Logs	<code>grep -i error file</code>	Search logs
Text	<code>awk '{print \$1}'</code>	First column
Text	<code>sed 's/a/b/g'</code>	Replace all
Text	<code>grep -rn pattern .</code>	Recursive search
Service	<code>systemctl status svc</code>	Service info
Service	<code>systemctl enable svc</code>	Auto-start
Users	<code>useradd -m user</code>	Create user
Users	<code>usermod -aG grp user</code>	Add to group
Scripting	<code>set -euo pipefail</code>	Strict mode
Scripting	<code>bash -x script.sh</code>	Debug mode

Focus areas for DevOps interviews: permissions, process management, networking, logs, shell scripting, and troubleshooting workflows. Theory matters less than being able to walk through a real incident step by step.